ARQUITECTURA DE SOFTWARE · UNIVERSIDAD SERGIO ARBOLEDA · 2026-3

Proyecto Matera · Fase 2

Del sensor al dominio: arquitectura hexagonal (puertos y adaptadores)

Modalidad: grupos del proyecto | **Tecnología:** Flask (Python) o Spring Boot (Java) | **Entrega:** Entrega 2

1. De dónde venimos

En la fase anterior construyeron una aplicación web tipo API que recibe **humedad, nivel de luz, temperatura y especie** de una planta y valida su estado contra una tabla de referencia (CSV o base de datos). La exigencia central fue separar responsabilidades por capas, aplicar los principios SOLID y escribir una lógica de negocio que no dependa de cómo se obtienen los datos. El front va separado del back, consume la API con peticiones asíncronas y no se genera desde el servidor.

Esta fase pone a prueba esa promesa. Hasta ahora los datos llegaban desde un formulario. A partir de ahora también llegarán desde la matera: un ESP32 que reporta lecturas cada pocos segundos. Más adelante vendrán una app móvil, la gestión de usuarios y la gamificación de la planta. Si su lógica de negocio de verdad es agnóstica, nada de eso debería obligarlos a reescribirla.

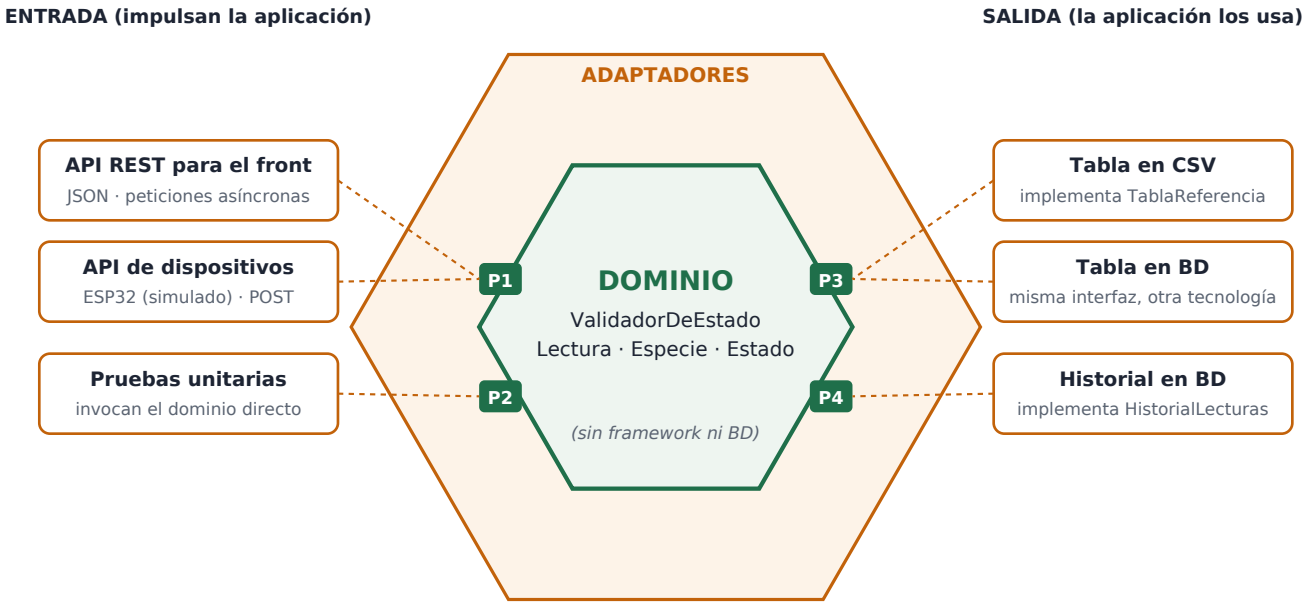
En este periodo ya trabajaron autenticación con **JWT**. En esta fase la incorporan al proyecto y la usan como caso de estudio: la seguridad también es tecnología, así que también vive afuera del dominio.

2. El marco: arquitectura hexagonal

La arquitectura hexagonal (Alistair Cockburn, 2005), también llamada *puertos y adaptadores*, organiza la aplicación alrededor de un núcleo de dominio que no sabe nada del mundo exterior. Todo lo que es tecnología (HTTP, frameworks, archivos, bases de datos, dispositivos) queda afuera y se conecta por interfaces que el propio dominio define.

Concepto	Qué es	En la matera
Dominio	Reglas de negocio puras. Sin imports de framework ni de infraestructura.	Validar si una lectura está en rango para la especie y calcular el estado de la planta.
Puerto de entrada	Interfaz que expone un caso de uso a quien quiera invocarlo.	*RegistrarLecturaPort*, *ConsultarEstadoPort*.
Puerto de salida	Interfaz que el dominio necesita y que otro implementa.	*TablaReferenciaPort*, *HistorialLecturasPort*.
Adaptador de entrada	Traduce un estímulo externo a una llamada al puerto.	Controlador REST del front, endpoint del dispositivo, pruebas unitarias.
Adaptador de salida	Implementa un puerto de salida con una tecnología concreta.	Tabla de referencia en CSV, tabla en BD, historial en BD.

Concepto	Qué es	En la matera
Seguridad (JWT)	Verificar y emitir tokens es infraestructura: filtro o decorador de entrada y librería JWT detrás de un puerto.	El filtro valida el token y entrega al caso de uso un *UsuarioId*, nunca el token. *EmisorTokenPort* lo implementa la librería JWT.
Composición	Único lugar donde se decide qué adaptador se conecta a qué puerto.	*main* / *create_app()* en Flask, clases *@Configuration* en Spring.



Las dependencias apuntan hacia adentro: los adaptadores conocen los puertos; el dominio no conoce a los adaptadores.

Figura 1. Vista de referencia. **P1** RegistrarLecturaPort · **P2** ConsultarEstadoPort · **P3** TablaReferenciaPort · **P4** HistorialLecturasPort. Los nombres son sugeridos; su diseño puede tener otros puertos si los justifica.

Relación con SOLID

- **Inversión de dependencias (D):** es el corazón del patrón. El dominio declara la interfaz; la infraestructura la implementa.
- **Abierto/cerrado (O):** agregar el ESP32 o cambiar CSV por BD se hace agregando un adaptador, no modificando el dominio.
- **Segregación de interfaces (I):** puertos pequeños y específicos, no un "Repositorio" que lo hace todo.
- **Responsabilidad única (S):** un adaptador traduce; no valida reglas de negocio.

3. Lo que deben construir

#	Requisito	Detalle
R1	Conservar la validación	La validación de la fase anterior sigue funcionando desde el front, ahora detrás de un puerto de entrada.
R2	Adaptador de dispositivos	Endpoint para que un dispositivo reporte lecturas, por ejemplo *POST /api/dispositivos/{id}/lecturas*. Cada lectura trae identificador del dispositivo, marca de tiempo y los valores medidos.

#	Requisito	Detalle
R3	Simulador del ESP32	Script independiente (Python o Java) que envía lecturas periódicas a su API. Intervalo configurable. Debe generar también valores fuera de rango y algún dato inválido, para probar la robustez del sistema. No se requiere hardware.
R4	Tabla de referencia intercambiable	Dos implementaciones del mismo puerto (CSV y BD). Se elige cuál usar por configuración, sin tocar código del dominio.
R5	Historial de lecturas	Persistir las lecturas recibidas y consultar la más reciente y el historial por dispositivo o planta.
R6	Front asíncrono	El front separado muestra el estado actual de la planta y se actualiza solo (consulta periódica a la API). No se genera desde el back.
R7	Autenticación con JWT	Registro e inicio de sesión de usuarios; cada planta pertenece a un usuario y solo él consulta su estado e historial. Los endpoints del front exigen un JWT de usuario. Los dispositivos se autentican con su propio token, con un alcance limitado a reportar lecturas de su planta. El simulador del ESP32 debe enviar ese token.

4. Restricciones de arquitectura (no negociables)

- El paquete o módulo de dominio **no importa** nada de Flask, Spring, HTTP, CSV, SQL ni JSON.
- Los puertos los define el dominio: interfaces en Java, clases abstractas o *Protocol* en Python.
- Las dependencias se conectan en **un solo punto de composición**. Ningún adaptador se instancia dentro del dominio.
- **Prueba de cambio:** el commit que agrega el adaptador de dispositivos y el simulador **no modifica ningún archivo del dominio**. Si lo modifica, deben explicar por qué en el documento de arquitectura.
- El dominio **no conoce JWT**: no importa la librería de tokens ni recibe cabeceras HTTP. La verificación ocurre en el adaptador de entrada (filtro de Spring Security o decorador en Flask) y el caso de uso recibe una identidad ya verificada. La regla de negocio "*un usuario solo ve sus plantas*" sí es del dominio.
- El dominio tiene pruebas unitarias que corren **sin levantar el servidor** y sin archivos ni base de datos (usen dobles de prueba para los puertos de salida).

Estructura de paquetes sugerida

Flask (Python)

```
matera/
  dominio/
    modelo.py      # Lectura, Especie, Estado
    validador.py
    puertos/
      entrada.py
      salida.py
  adaptadores/
    entrada/
      api_front.py
      api_dispositivos.py
    salida/
      tabla_csv.py
      tabla_bd.py
      historial_bd.py
  app.py             # composición
simulador_esp32.py
tests/
```

Spring Boot (Java)

```
co.edu.usa.matera
  dominio
    modelo    // Lectura, Especie, Estado
    servicio      // ValidadorDeEstado
    puerto.entrada
    puerto.salida
  adaptador
    entrada.rest
    entrada.dispositivo
    salida.csv
    salida.jpa
  config          // @Configuration
simulador/
src/test/java/
```

5. Sobre el uso de inteligencia artificial

Pueden usar asistentes de IA para escribir código, depurar y explorar ideas. Lo que **no** pueden delegar es la descripción de la arquitectura: es la evidencia de que ustedes entienden lo que construyeron. Por eso:

- No suban este enunciado a un modelo de lenguaje para que les redacte el documento de arquitectura. Ese documento lo escriben ustedes.
- El diagrama hexagonal de su proyecto debe corresponder a **su** código: nombres reales de clases, puertos y adaptadores.
- En la sustentación se les hará una pregunta de cambio sin computador (por ejemplo: "*mañana las lecturas llegan por MQTT en lugar de HTTP, ¿qué archivos tocan?*" o "*la universidad exige iniciar sesión con OAuth en vez de usuario y contraseña, ¿qué cambia?*"). Cualquier integrante debe poder responderla.

6. Entregables

- **Repositorio** con el código, el simulador y un README que explique cómo ejecutar el back, el front y el simulador.
- **Documento de arquitectura** (máximo 4 páginas) con: diagrama hexagonal propio; tabla de puertos y sus adaptadores; explicación de dónde se emite y dónde se verifica el JWT; justificación de al menos tres principios SOLID con ejemplos de su código; enlace al commit de la prueba de cambio y análisis de qué cambiaría si las lecturas llegaran por MQTT.
- **Sustentación** en clase: demostración con el simulador enviando lecturas en vivo, cambio de CSV a BD por configuración, intento de acceso a una planta ajena (debe fallar) y respuesta a la pregunta de cambio.

7. Rúbrica

Criterio	Peso	Nivel esperado
Aislamiento del dominio	20%	El dominio no tiene dependencias de framework ni de infraestructura y se prueba de forma aislada.
Puertos y adaptadores	20%	Puertos definidos por el dominio, pequeños y con nombres de negocio. Adaptadores que solo traducen.
Prueba de cambio	15%	El adaptador de dispositivos y el simulador entran sin modificar el dominio; CSV y BD se intercambian por configuración.
Pruebas del dominio	10%	Pruebas unitarias con dobles de prueba que cubren lecturas en rango, fuera de rango e inválidas.
Seguridad con JWT	10%	Tokens de usuario y de dispositivo con alcances distintos; la verificación vive en el adaptador y el dominio aplica la regla de propiedad.
Front asíncrono y simulador	10%	El front se actualiza solo; el simulador genera datos realistas y casos de error.
Documento y sustentación	15%	El documento corresponde al código, justifica decisiones y todos los integrantes responden la pregunta de cambio.

8. Hacia dónde va la matera

En las siguientes fases llegarán el ESP32 real, la app móvil y la gamificación de la planta. En una arquitectura hexagonal bien hecha, cada una de esas piezas es **un adaptador nuevo** o **un caso de uso nuevo**, nunca una reescritura del dominio. Lo que construyan en esta fase es la base que decide cuánto les va a costar todo lo demás.

Declaración de uso de inteligencia artificial

> Este enunciado fue elaborado por el docente con apoyo de **Claude**, el asistente de inteligencia artificial de Anthropic. El docente definió los objetivos de aprendizaje, los principios del proyecto, los requisitos, las restricciones de arquitectura y los criterios de evaluación. Claude apoyó la redacción, la organización del documento y la elaboración del diagrama.
>
> El docente revisó y ajustó todo el contenido, y es el único responsable de él. Hacemos esta declaración por la misma razón por la que les pedimos transparencia en su proyecto: usar IA es legítimo cuando se declara y cuando quien firma entiende y respalda lo que entrega.